1081. Smallest Subsequence of Distinct Characters

Solved

Medium | Topics | Companies | Hint

Given a string `s`, return *the lexicographically smallest subsequence of* `s` *that contains all the distinct characters of* `s` *exactly once.*

Example 1:

```
Input: s = "bcabc"
Output: "abc"
```

Example 2:

```
Input: s = "cbacdcbc"
Output: "acdb"
```

Constraints:

- `1 <= s.length <= 1000`
- `s` consists of lowercase English letters.

Note: This question is the same as 316: https://leetcode.com/problems/remove-duplicate-letters/

```java
class Solution {
    public String smallestSubsequence(String s) {
        int[] charFrequency = new int[26];
        boolean[] inStack = new boolean[26];
        Arrays.fill(inStack, false);
        for (char c : s.toCharArray()) {
            charFrequency[c - 'a']++;
        }
        Deque<Character> monotonicStack = new ArrayDeque<>();
        for (char c : s.toCharArray()) {
            if (inStack[c - 'a'] == true) {
                charFrequency[c - 'a']--;
                continue;
            }
            while (!monotonicStack.isEmpty() && monotonicStack.peekLast() >
c) {
                if (charFrequency[monotonicStack.peekLast() - 'a'] > 0) {
                    char x = monotonicStack.pollLast();
                    inStack[x - 'a'] = false;
                } else {
                    break;
                }
            }
            charFrequency[c - 'a']--;
            inStack[c - 'a'] = true;
            monotonicStack.addLast(c);
        }
        StringBuilder result = new StringBuilder();
        while (!monotonicStack.isEmpty()) {
            result.append(monotonicStack.pollFirst());
        }
        return result.toString();

    }
}
```

→ Take two arrays: one for counting frequencies and another for checking inStack or not.

→ for every character:

* Check if it is already in stack. If yes, then decrement freq and continue.
* While stack is not empty and top value is greater than current char then:
  * If freq. of current char is > 0 then pop as it is not at correct position. Set false for inStack.
  * Else break from while.
* After while decrement value for char and true for inStack and add to stack.

→ Now generate result by polling from first to preserve the correct sequence.